TITLE

Dynamic Thermo-Elastic Mesh Simulation for Parallel Systems

Team Members: Allison and Alan

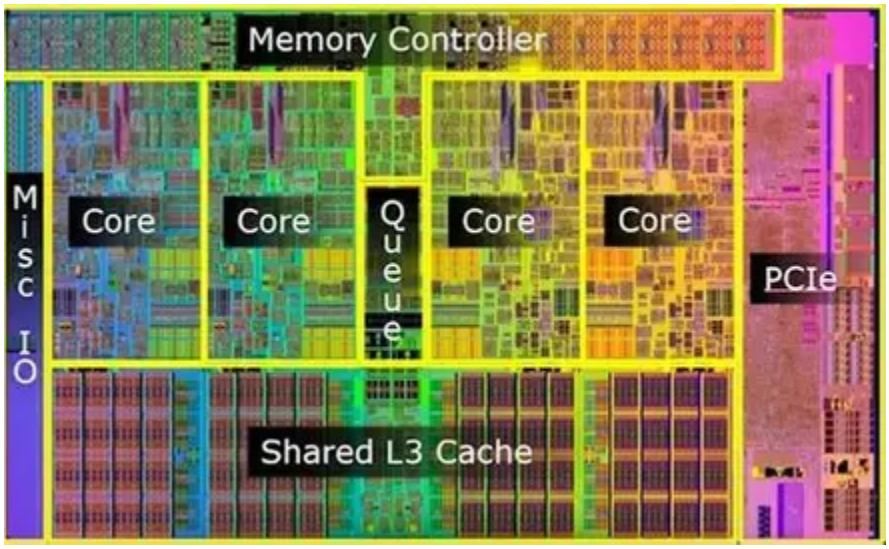
URL

[Placeholder: Link to your GitHub Pages or Andrew UserWeb project site]

SUMMARY

We will build a parallel simulator of thermal diffusion on a dynamic 2D mesh where elements deform based on temperature, inspired by shape-memory alloys. The simulation will model heat flow and structural feedback across a processor floorplan, parallelized using C++ and OpenMP/pthreads on multi-core CPU systems.

BACKGROUND



Our project focuses on simulating how heat spreads and causes stress in a processor chip, taking inspiration from materials like shape-memory alloys (SMAs) that deform predictably with temperature. While real chips don't actively deform like SMAs, thermal hotspots still cause small expansions and mechanical stress in silicon and metal layers, which can affect heat flow and reliability. Our goal is to model this thermomechanical coupling in a simplified but realistic way.

We represent the chip as a 2D mesh of cells, where each cell corresponds to a functional unit, such as an ALU, cache block, or interconnect region. Each cell has a temperature, generates heat based on its activity, and conducts heat to its neighbors. The **core computation** involves updating each cell's temperature from its neighbors and its own power generation, then computing how much it expands due to heating. The expansion slightly changes the effective distance to neighboring cells, which affects heat conduction in the next timestep.

This simulation is compute-intensive because each cell must repeatedly update both temperature and strain, and these updates depend on the current state of neighboring cells. The problem is naturally suited to **parallelism**: updates for different cells can be computed simultaneously since each timestep only depends on the previous values. However, as hotspots form, some regions may require more computation than others, creating potential load imbalance. Parallelizing over cells or blocks of cells allows multiple cores to work together efficiently, speeding up the simulation while handling the evolving workload.

THE CHALLENGE

This problem is challenging due to the **dynamic feedback loop** between heat and geometry.

- **Workload Characteristics:** The simulation involves high spatial locality but faces **divergent execution** if certain regions (hotspots) require more complex deformation or topology updates than others.
- **Dependencies:** There is a strict temporal dependency; geometry updates depend on the new temperature state, and the next temperature state depends on the updated geometry.
- **System Constraints:** As elements move or expand, memory access patterns that were once predictable (in a static grid) may become irregular, leading to cache misses and potential load imbalances.

RESOURCES

- **Computers:** We will use the GHC lab machines and/or the PSC cluster for multi-core CPU testing.
- **Code Base:** We are starting from scratch to build a custom C++/OpenMP engine.
- **References:** We are utilizing *HotSpot* for architecture-level thermal modeling concepts and recent research on parallel finite-difference heat equations.

GOALS AND DELIVERABLES

MUST ACHIEVE

1. 2D Mesh Simulator

Cell characteristics:

- **Temperature (T):** Current temperature of the cell.
- **Heat capacity (c):** How much energy it takes to raise the cell's temperature—can vary by type (ALU, cache, interconnect).
- **Power generation (p):** Amount of heat the cell produces per timestep, reflecting activity level.
- **Thermal conductivity (k):** Rate at which the cell exchanges heat with each neighbor. Can be directional if desired (e.g., vertical vs horizontal conduction).
- **Neighbors (N_i):** List of adjacent cells or a fixed stencil (4-way or 8-way).

Customizable aspects:

- Different cell types with distinct **c, p, k** values.
- **Cell type tagging:** ALU, cache, memory, interconnect → different thermal/mechanical parameters.
- Non-uniform mesh spacing (some cells larger to represent bigger blocks).
- Boundary conditions: fixed temperature, heat sink, or insulated edges.

---

2. Coupled Deformation

Goal: Model thermomechanical feedback: cells expand/contract depending on temperature, affecting thermal coupling.

Cell characteristics:

- **Coefficient of thermal expansion (α):** Determines how much the cell expands per unit temperature rise.
- **Geometry / size:** Width/height or area of the cell (affects neighbor distances and thus conductance).
- **Strain / displacement (ε):** Optional: track cumulative deformation.
- **Neighbor conductances (k_ij):** Adjust dynamically based on expansion/contraction.

Customizable aspects:

- Different expansion coefficients per material type.
- Adjustable method of conductance scaling (linear, nonlinear, capped, or weighted by distance).
- Option to track cumulative strain for “reliability-aware” simulations.

---

3. Parallel Multi-threaded Implementation

Goal: Accelerate simulation by updating many cells simultaneously.

Considerations per cell:

- Updates for **T and strain** are independent within a timestep (based on previous step’s values).
- **Neighbor access pattern**: for conductance update, a cell may read adjacent cells’ strain; avoid race conditions via double-buffering or per-thread local copies.
- **Workload weight:** Some cells produce more heat, which causes larger temperature changes, which means more compute in some areas.

Customizable aspects:

- Choice of threading library: OpenMP, pthreads, or hybrid.
- Thread assignment strategy: static blocks vs. dynamic work queue.
- Adjustable number of threads for scaling experiments.

---

4. Performance Evaluation

Goal: Measure how efficiently the simulation scales across multiple threads and explore the effects of different parallelization strategies. The focus is on achieving high speedup and maintaining good load balance, rather than on detailed thermal correctness.

Metrics:

- **Speedup:** time1thread/timeNthreads\text{time}_{1\text{thread}} / \text{time}_{N\text{threads}}time1thread/timeNthreads
- **Parallel efficiency:** How close observed scaling is to ideal linear speedup.
- **Load balance:** Track per-thread execution time or work distribution to identify imbalance caused by hotspots or irregular mesh layouts.

Customizable aspects (for achievable experiments):

- **Mesh size:** e.g., 100×100, 500×500 cells
- **Power patterns:** uniform, a few hotspots, or random loads
- **Number of threads and thread scheduling strategies** (static vs. simple dynamic partitioning)

Analysis approach:

- Break down execution time into compute, synchronization, and geometry update phases.
- Compare speedup across thread counts to evaluate how close the simulation gets to ideal scaling.
- Identify major bottlenecks, such as memory contention, synchronization barriers, or uneven workloads.
- Present graphs showing speedup vs. thread count, per-thread load distribution, and time breakdowns.

Potential Solutions

- Overlap **temperature update and geometry update** using double buffering and fine-grained synchronization, so some threads start computing the next step while others finish current mesh updates.
- Experiment with **non-blocking barriers** or pipelined updates to reduce waiting time between timesteps.

NICE TO ACHIEVE

1. **Interactive / Live Simulation**
- Allow the simulator to run in near real-time, updating heatmaps and strain dynamically as power inputs change.
- Provide a live visualization of the mesh deforming or the heat flow changing over time.
2. **Customizable Inputs & Scenarios**
- Write a program or interface to adjust power profiles on-the-fly, e.g., simulate a hotspot suddenly powering up.

- Allow users to set cell material properties, expansion coefficients, or thermal conductances dynamically.
- Include **dynamic load injection** to test extreme or non-uniform scenarios.

3. **Flexible Mesh / Floorplan Import**

- Support importing arbitrary processor floorplans, e.g., from JSON, CSV, or standard CAD formats.
- Map imported floorplans to the mesh, assigning thermal properties to each block automatically.

4. **Enhanced Visualization & Output**

- Generate animated heatmaps or videos showing thermal propagation and mesh deformation over time.
- Produce performance dashboards combining heatmaps, strain, and thread utilization for parallel runs.

PLATFORM CHOICE

We have chosen **C++ with OpenMP/pthreads** for shared-memory parallelism. This platform allows for fine-grained control over memory layout (Structure-of-Arrays vs. Array-of-Structures), which is critical when managing the cache locality of a mesh that is physically moving in memory.

SCHEDULE

Week	Task/Milestone
Mar 23–25	Finalize design and submit Proposal (Due Mar 25).
Mar 26–31	Implement serial static mesh and verify heat diffusion correctness.
Apr 1–7	Implement temperature-dependent expansion and initial OpenMP parallel loops.
Apr 8–14	**Milestone:** Working parallel prototype; measure initial speedups (Due Apr 14).
Apr 15–21	Refine data structures for unstructured mesh; experiment with dynamic scheduling.

Apr 22–28	Run final experiments across various mesh sizes and thread counts.
Apr 29–30	Final Report writing and Poster preparation (Due Apr 30).
May 1	**Poster Session** (15-418: 8:30am